Gestiunea Securitatii

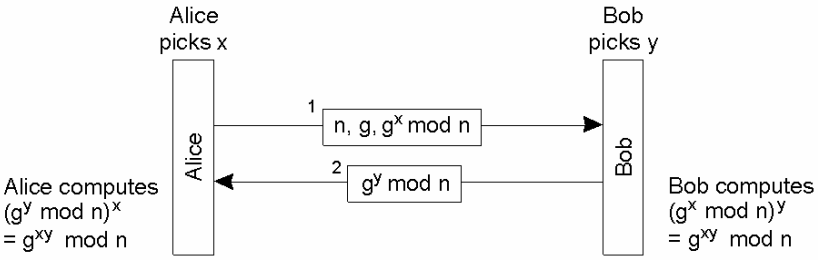
Aceasta sectiune acopera urmatoarele subiecte:
Gestiunea cheilor, adica stabilirea cheilor si distributia cheilor
Gestiunea sigura a grupurilor (Secure Group Management), mai precis schimbarea si distributia cheii de grup la adaugarea unui nou membru in grup si la scoaterea unui membru din grup.
Gestiunea autorizarii, adica delegarea de permisiuni de la un utilizator autentificat (principal) catre o entitate care actioneaza in numele sau.

Stabilirea cheilor

1. Cheile secrete (folosite in criptografia cu chei simetrice) pot fi generate in doua moduri:
- De catre o a treia parte, de incredere, un centru de distributie a cheilor (KDC – Kay Distribution Center)
- Fara a treia parte; de exemplu folosind protocolul Diffie-Hellman cu urmatorii pasi:
Alice si Bob trebuie sa cada de acord asupra a doua numere foarte mari *n* si *g*. Ambele numere pot fi publice.
Alice alege un numar foarte mare *x*, pe care il pastreaza secret. Bob face acelasi lucru pentru *y*.
Pentru Alice:

x	este cheia privata
$g^x \bmod n$	este cheia publica
$K_{A,B} = g^{xy} \bmod n$	este cheia secreta partajata cu Bob

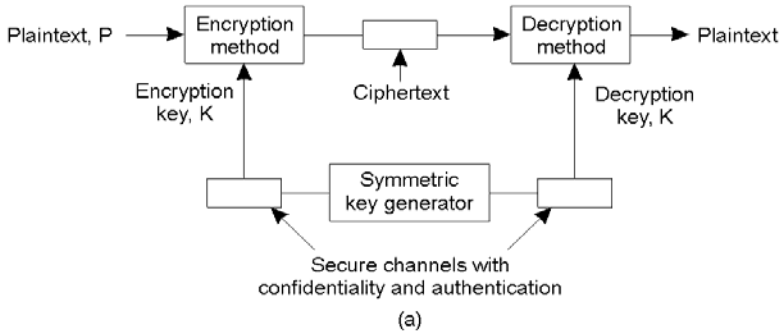
Generarea cheii secrete este rezumata in figura urmatoare.



2. Cheile publice si private (folosite in criptografia cu chei publice) pot fi generate de catre o autoritate de certificare (CA = Certificate Authority).

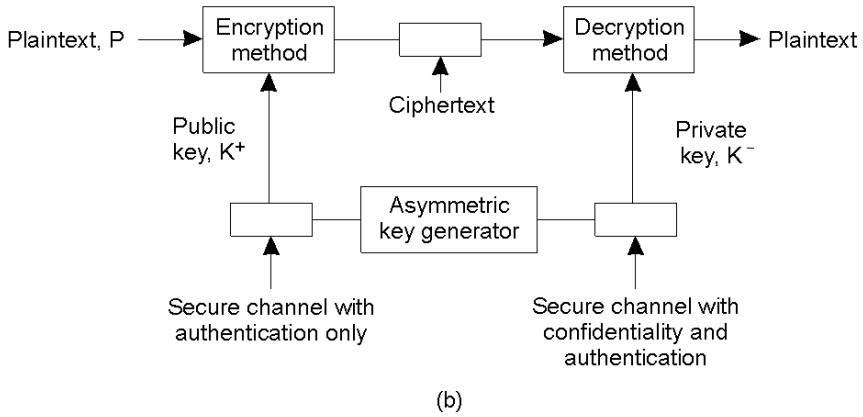
Distributia cheilor secrete

Cheile secrete trebuie cunoscute doar de persoanele autorizate. De aceea, distributia cheilor secrete foloseste cai autentificate (astfel ca receptorii stiu de la cine provin cheile) si confidentiale (astfel incat cheile nu sunt accesibile persoanelor neautorizate). Cele mai uzuale cai sunt: telefonul, posta, valiza diplomatica, contactul direct.



Distributia cheilor publice

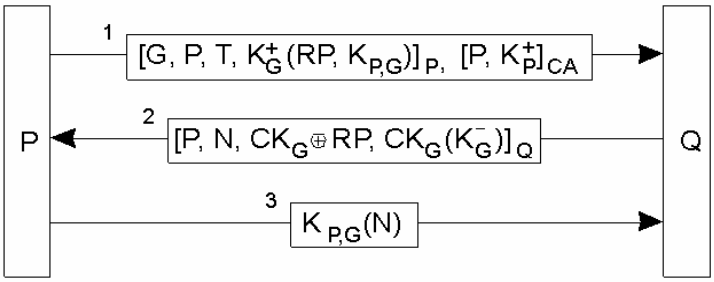
Cheia publica este transmisa prin canale autentificate (trebuie doar ca transmitatorul sa fie cunoscut dar nutrebuie secretizata transmisia). Forma de difuzare a cheii este Certificatul, care este semnat de Autoritatea de Certificare.



Cheia privata se distribuie prin canale confidentiale si autentificate, avand acelsi regim ca cheile secrete din criptografia cu chei simetrice (vezi figura de mai sus).

Gestiunea sigura a grupurilor

Incepem prezentarea gestiunii sigure a grupurilor cu admiterea unui nou membru in grup.



Presupunec ca grupul foloseste urmatorul set de chei:

K^+_G / K^-_G perechea de chei publica / privata, pentru a comunica cu alte procese din afara grupului
CK_G cheie secreta folosita in interiorul grupului

Problema: pentru admiterea unui proces P in grup se excuta urmatorul protocol.
1 - Procesul P trimite o cerere de alaturare la grup, semnata de el (mesaj 1) care include:
T – timpul curent local lui P
RP – Reply Pad – folosit la alcatuirea raspunsului
$K_{P,G}$ – cheie secreta partajata intre P si G
si un certificat semnat de CA cu cheia sa publica K^+_P
2 - Un proces oarecare din grup, Q autentifica P si verifica daca P poate fi admis ca membru in grup. Dupa terminarea cu succes a verificarii, Q transmite mesajul semnat 2, de admitere in grup, care contine :
N – Nonce
CK_G – cheia secreta a grupului G combinata cu RP
K^-_G – cheia privata a grupului criptata cu cheia secreta a grupului CK_G
Obs: in mesajul 2, CK_G este criptat cu RP, nu cu K^+_P pentru a nu compromite cheia CK_G daca K^-_P este compromisa.
3 - P autentifica Q si transmite inapoi N criptat cu cheia secreta partajata $K_{P,G}$.

Managementul cheilor pentru comunicarea de grup

Neajunsul metodei prezentate este ca noul venit, P poate vedea mesaje anterioare admiterii sale in grup. Uzual, cheia de grup se schimba la modificarea componentei grupurilor astfel incat un nou membru sa nu poate intelege mesajele anterioare intrarii sale in grup, iar un fost membru (care a iesit sau a fost exclus din grup) sa nu poate intelege mesajele transmise dupa ce a parasit grupul.
Distribuirea unei noi chei de grup este banala la alaturarea unui nou membru: noua cheie se transmite multicast, criptata cu cheia veche. Distribuirea este foarte complicata la iesirea unui membru. Descriem succint o schema nescalabila, bazata pe folositea unui centru de distributie a cheilor, KDC. KDC are cate o cheie secreta cu fiecare membru din grup. KDC genereaza noua cheie si alcatuieste un mesaj cu n inregistrari, fiecare inregistrare constand din noua cheie criptata cu cheia secreta partajata de KDC si de un membru din grup (KDC face n criptari). KDC transmite multicast mesajul (mesajul este mare). Fiecare membru receptioneaza mesajul si retine partea criptata cu cheia sa. Prin decriptarea ei, fiecare membru intra in posesia noii chei de grup.

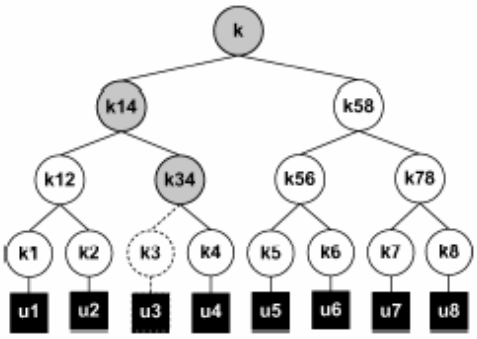
Protocoale pentru managementul cheilor de grup

Management cheilor are mai multe roluri :
Identificarea si autentificarea membrilor grupului astfel incat sa se poate evita atacurile de tip mascarada (un intrus "se da" drept membru al grupului).
Controlul accesului (validarea operatiei de alaturare a unui membru la grup inainte de realizarea acesteia).
Generarea, distributia si instalarea cheilor (o atentie deosebita se acorda cerintei ca noile chei sa fie independente de cele vechi).
In cele ce urmeaza ne referim la ultimul din aceste roluri. Sunt cunoscute mai multe categorii de protocoale:

Centralizate - Exista un singur KDC. Protocoalele urmaresc sa minimizeze cerintele de memorie, puterea de calcul consumata la client si server si folosirea largimii de banda.
Descentralizate - Sunt mai multe KDC iar grupul este impartit in mai multe subgrupuri. Se urmareste evitarea concentrarii incarcarii intr-un singur punct.
Distribuite - nu exista un KDC controlul accesului si generarea cheii fiind facute de membrii grupului.

Management centralizat – LKH (Logical Key Hierarchy)

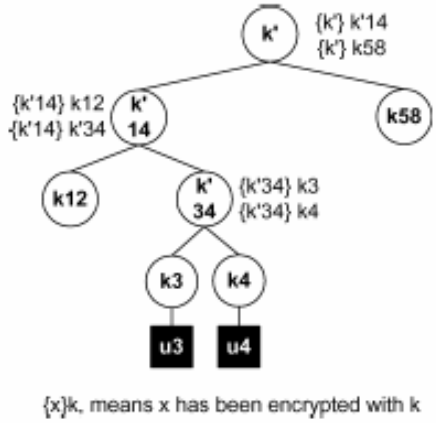
Propus Wong, Wallner
KDC pastreaza chei organizate logic ca un arbore de chei (vezi figura).



Radacina corespunde cheii de grup (cu care se cripteaza mesajele). Fiecare nod corespunde unei chei de criptare a altor chei, KEK (Key Encryption Key). Fiecare frunza este asociata cu cheia KEK a unui membru din grup.
Fiecare membru are cheia KEK din frunza si cheile KEK din calea de la el la radacina (in total $\log_2 n + 1$ chei, $\log_2 n$ = adancime arbore balansat).
De ex: u_1 cunoaste cheile k_1, k_{12}, k_{14} si k.

LKH (Logical Key Hierarchy) – nou membru

Membrul nou este asociat cu o frunza si inclus in arbore.
Cheia radacinii si toate cheile nodurilor pe calea la noul nod la radacina trebuie



schimbate. Pentru asta, KDC compune mesaj cu noile KEK; un KEK este criptat cu fiecare din cheile fiilor nodului respectiv (lungimea mesajului este $2\log_2 n$). Mesajul este

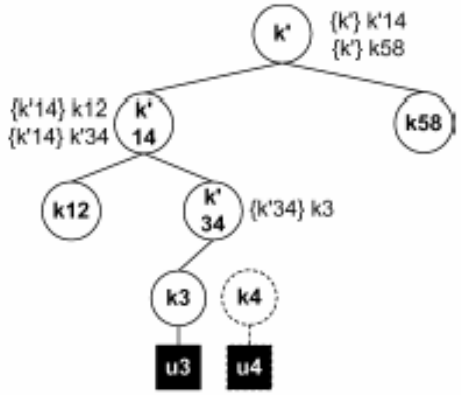
transmis multicast membrilor grupului. Fiecare membru retine partea care-l intereseaza si face decriptarile necesare pentru a afla cheia de criptare a datelor si cheile KEK modificate aflate pe calea de la cheia sa la radacina.

De exemplu: nodul u3 primeste cheia k3 si este adaugat la nodul k34. Cheile k34, k14 si k trebuie schimbate. Ca urmare,

- KDC genereaza noile chei k'34, k'14 si k'
- Cripteaza fiecare cheie cu cheile fiilor lor; de ex. k' este criptat cu k'14 si k58; celelalte chei sunt criptate conform figurii anterioare
- Compune un mesaj de lungime 2*3
- Transmite mesajul multicast
- Fiecare membru calculeaza noul set de chei prin realizarea decriptarilor necesare; astfel, u4
  - foloseste k4 si afla k'34,
  - foloseste k'34 si afla k'14,
  - foloseste k'14 si afla k'
- u5 face doar o decriptare - foloseste k58 si afla k'

LKH (Logical Key Hierarchy) – eliminare membru

La eliminarea unui nod, operatiile sunt similare. De ex. la eliminarea lui u_4, noile chei sunt criptate cu cheile KEK ale fiilor cu exceptia lui k'_{34} care este criptat doar cu cheia fiului ramas, k_3 nu si cu cheia k4 a lui u4.



O imbunatatire adusa de Waldvogel este protocol LKH+ care foloseste o functie one-way pentru a calcula noile chei pornind de la cheile compromise. Aceasta schimbare nu respecta cerinta ca cheile noi sa fie independente de cheile vechi.

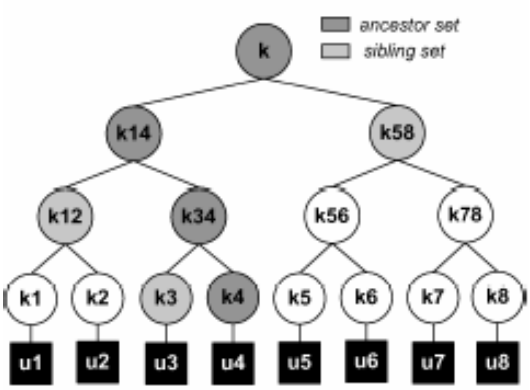
OFT - One-way Function Tree

Propus de McGrew si Sherman.
Schema reduce lungimea mesajelor la $\log_2 n$. Ideea este de a calcula cheia unui nod prin "orbirea" cheilor fiilor si mixarea dupa formula urmatoare:

$$K_i = f(g(k_{left(i)}), g(k_{right(i)}))$$

unde left(i), right(i) – sunt fiii lui i
g – este o functie one-way de "orbire"

f – este o functie de mixare.



In aceasta metoda, fiecare nod are asociate doua seturi de noduri asociate:

- Stramosi (ancestors) – nodurile de la el la radacina
- Rude (siblings) – fratii nodurilor din setul Stramosi

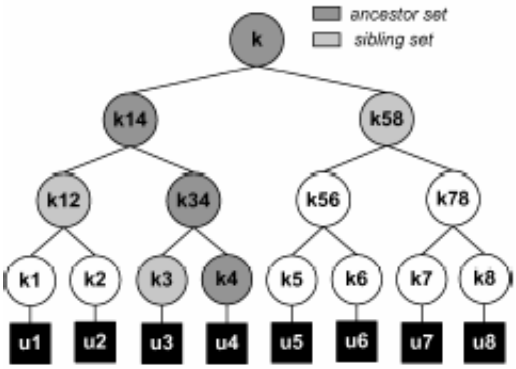
De ex: seturile pentru u4 sunt reprezentate in figura prin gri inchis (stramosi) si gri deschis (rude).

Fiecare membru pastreaza $\log_2 n+1$ chei: cheia sa, cheia orbita a fratelui si cheile orbite ale rudelor. De exemplu, u_4 pastreaza 4 chei:

Cheia sa (k_4)
Cheia orbita a fratelui $g(k_3)$
Cheile orbite ale Rudelor $g(k_{12})$, $g(k_{58})$

Din aceste info, u_4 poate genera cheile KEK aflate pe calea la radacina si cheia de criptare a datelor:

$k_{34} = f(g(k_3), g(k_4))$
$k_{14} = f(g(K_{12}), g(k_{34}))$
$k = f(g(k_{58}), g(k_{14}))$



OFT – nod nou

Consideram cazul incare u_3 se alatura grupului. In aceasta situatie, trebuie modificate k_{34}, k_{14}, si k. KDC transmite 3 chei orbite $g(k_3)$, $g(k'_{34})$, $g(k'_{14})$ criptate respectiv cu k_4, k_{12} si k_{58} adica:

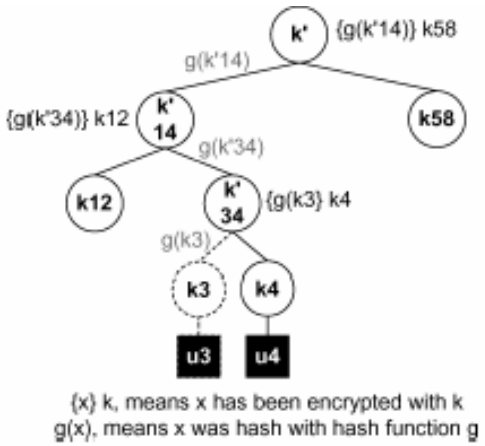
$\{g(k_3)\}k_4$, $\{g(k'_{34})\}k_{12}$, $\{g(k'_{14})\}k_{58}$

Noile chei pot fi calculate de fiecare membru al grupului (care are cheia respectiva pentru decriptare) aplicand operatii de decriptare. De exemplu, u4 executa urmatoarele operatii:

afla $g(k_3)$ prin decriptare si calculeaza $k'_{34} = f(g(k_3), g(k_4))$

calculeaza $g(k'_{34})$ si apoi $k'_{14} = f(g(k_{12}), g(k'_{34}))$

calculeaza $g(k'_{14})$ si apoi $k' = f(g(k'_{14}), g(k_{58}))$

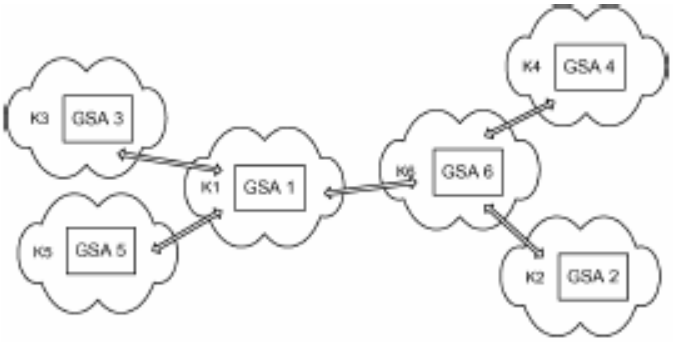


Protocoale descentralizate - Iotus

Propus de Mittra.

In Iotus, grupul impartit in sub-grupuri, fiecare gestionat de un GSA – Group Security Agent. GSA formeaza,la randul lor, un grup de nivel inalt gestionat de Group Security Controller.

Cu aceasta schema, sub-grupurile au chei diferite, independente astfel incat modificarea unei chei de sub-grup afecteaza un numar mai mic de membri. Dezavantajul este ca transmiterea mesajelor de la un grup la altul necesita translatarea de la o cheie de criptare la alta.



DEP - Dual-Encryption Protocol

Propus de Dondeti.

Si aceasta schema imparte grupul in subgrupuri, dar cheia de grup este unica, eliminandu-se astfel neajunsul translatarii de la o cheie la alta.
Fiecare sub-grup este controlat de un SGM – Sub-Group Manager. Exista trei grupuri de KEK:

KEK_{i1} – partajata de SGM_i si membrii grupului i
KEK_{i2} – partajata de **Group Controller - GC** si membrii grupului i fara SGM_i
KEK_{i3} – partajat de GC si SGM_i

DEK – Data Encryptioin Key este una singura pentru grup (asa cum spuneam anterior).
Distributia unei chei DEK se face astfel:

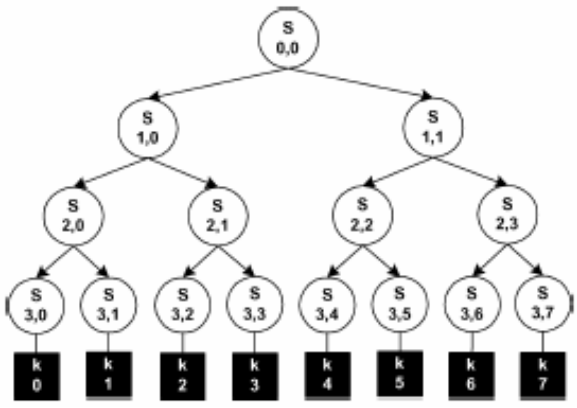
GC transmite pachete $\{\{DEK\}KEK_{i2}\}KEK_{i3}$
SGMi recupereaza $\{DEK\}KEK_{i2}$; el nu cunoaste KEK_{i2} deci nu poate afla cheia DEK.
SGMi Cripteaza mesajul $\{\{DEK\}KEK_{i2}\}KEK_{i1}$ si-l trimite subgrupului i
Fiecare membru al subgrupului i decripteaza de doua ori si obtine DEK.

La schimbarea componenței subgrupului i, SGM_i schimba KEK_{i1} si-l transmite membrilor actuali ai grupului. Urmeaza schimbarea DEK si distributia ei dupa modelul prezentat.
Ca dezavantaj, membrii exclusi pot primi mesajele grupului pana la terminiarea difuzarii DEK.

MARKS

Propus de Briscoe.
Acest model se bazeaza pe impartirea timpului in intervalle. Protocolul foloseste o cheie de criptare distincta pentru fiecare interval. Cheile sunt frunze intr-un arbore hash generat dintr-o singura sursa,dupa algoritmul descris in continuare.
Notatie: $S_{i,j}$ inseamna nodul j la nivelul i al arborelui.
Se calculeaza adancimea D (conform numar total chei N = 2^D)
Se alege aleator radacina $S_{0,0}$
Se genereaza sursele intermediare *left* $S_{1,0}$ si *right* $S_{1,1}$

$S_{1,0}$ = b(ls(S0,0))
$S_{1,1}$ = b(rs(S0,0))
unde ls = left shift one bit
rs = right shift one bit
b = blinding function



Se genereaza similar celelalte nivele.

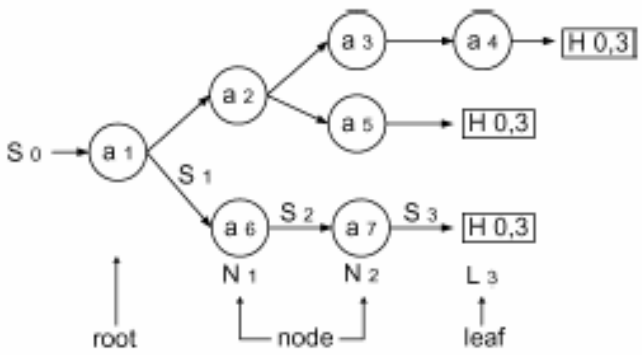
Calculul cheilor este distribuit utilizatorilor. De exemplu, un membru care participa la grup in intervalele 3-7 primeste sursele $S_{3,3}$ si $S_{1,1}$ si foloseste $S_{3,3}$ drept k3, iar din $S_{1,1}$ calculeaza k4,…,k7.

CS - Cipher Sequences

Propus de Moldva si Pannetrat.
In aceasta metoda, se foloseste o functie speciala f(S,a), numita Cipher Group, avand urmatoarele proprietati:

Exista o secventa de n elemente $a_1,\ldots, a_n$
Exista o secventa de n+1 elemente $S_0,\ldots S_n$
$S_i = f(S_{i-1},a_i)$ pentru i>0 si valoarea initiala S_0 si
Pentru fiecare cuplu (i,j) cu i<j exista o functie $h_{i,j}$ a.i. $S_i = h_{i,j}(S_j)$.

Grupul multicast este pus intr-un arbore cu sursa in radacina, membrii in frunze si nodurile interne ca elemente intermediare pentru transmiterea multicast. Fiecare nod Ni are o informatie asociata ai>0 si poate executa functia f.



Sa presupunem ca S0 este informatia de transmis multicast. Atunci cand nodul Ni primeste valoarea Sj de la parinte, el calculeaza Si = f(Sj, ai) si transmite Si copiilor sai din arbore. Frunzele cunosc functia $h_{0,n}$ astfel ca pot calcula valoarea initiala prin formula $S0 = h_{0,n}(Sn)$.

Intr-un aranjament descentralizat, frunza este compusa din mai multi membri (un subgrup) care impart aceeasi functie $h_{0,n}$. La schimbarea componenței subgrupului, nodul N_n primeste o noua valoare a'_n in locul vechii valori a_n. Toti membrii din frunza, cu exceptia membrului eliminat, primesc o noua functie $h'_{0,n}$ corespunzatoare noii valori a'_n astfel ca o noua cheie S0 poate fi transmisa membrilor grupului fara ca aceasta sa mai poate fi inteleasa de membrul eliminat.

Protocoale distribuite - Group Diffie-Hellman key exchange

Propus de Steiner.
Acest protocol este o generalizare a protocolului Diffie-Hellman pentru doi parteneri. Consideram ca grupul are n membri, fiecare membru i avand asociat un numar secret xi. Grupul stabileste doua numere prime (q si a) si incepe sa calculeze cheia de grup, in mai multe etape, trecand prin mai multe valori intermediare. Protocolul are n runde.

Primul membru calculeaza a^{x1} si paseaza valoarea obtinuta urmatorului membru. Acesta foloseste numarul sau secret, x2, calculeaza setul $\{a^{x1}, a^{x2}, a^{x1.x2}\}$ si il transmite urmatorului membru din grup.
Fiecare membru primeste setul de valori intermediare si genereaza un nou set folosind numarul sau secret. Un set generat de membrul **i** va avea **i** valori **intermediare** cu **i-1** exponenti si o valoare **cardinal** cu **i** exponenti. De exemplu, al patrulea membru primeste setul $\{a^{x2.x3}, a^{x1.x3}, a^{x1.x2}, a^{x1.x2.x3}\}$ si genereaza $\{a^{x2.x3.x4}, a^{x1.x3.x4}, a^{x1.x2.x4}, a^{x1.x2.x3}, a^{x1.x2.x3.x4}\}$ in care valoarea cardinal este $a^{x1.x2.x3.x4}$. Membrul n calculeaza cheia k din valoarea cardinal $k = a^{x1.x2.x3\ldots xn} \bmod q$, dupa care ridica toate valorile intermediare la valoarea sa secreta xn si transmite multicast tot setul cu exceptia cheii k. Fiecare membru extrage din set acea valoare intermediara care ii permite sa obtina cheia k prin ridicarea ei la valoarea sa secreta.

Octopus

Propus de Becker si Wille.
Este bazat tot pe protocolul Diffie-Hellman, dar are o alta abordare. Grupul este impartit in patru subgrupuri Ga, Gb, Gc, Gd cu liderii A, B, C si D (liderii nu apartin grupurilor, de exemplu A nu apartine lui Ga). Sunt parcurse urmatoarele etape.

1. A stabileste (Diffie-Hellman) o cheie ui cu fiecare membru i din Ga
   A calculeaza $K(Ga) = \Pi$ ui, cu iϵGa
   A calculeaza $Ia = a^{K(Ga)}$
   similar fac B, C, D.
2. A si B schimba valorile intermediare (Ia si Ib) si calculeaza $Iab = a^{K(GaUGb)}$
   C si D schimba valorile intermediare (Ic si Id) si calculeaza $Icd = a^{K(GcUGd)}$
3. A si C schimba valorile Iab, Icd si le folosesc drept valori secrete obtinand $a^{Iab.Icd}$
   Similar B si D.
4. A trimite fiecarui membru **j** al subgrupului sau valorile $a^{K(Ga\setminus\{uj\}UGb)}$ si a^{Icd}
   Fiecare membru calculeaza valoarea Iab si cheia $k = a^{Iab.Icd} \bmod q$.
   Similar procedeaza B, C si D

Numarul de runde este $2(n-4)/4 + 2$.

CKA - Conference Key Agreement

Propus de Becker si Wille.
Cheia este generata cu o functie de combinare f, utilizand formula:

$$K = f(N1, h(N2), \ldots, h(Nn))$$

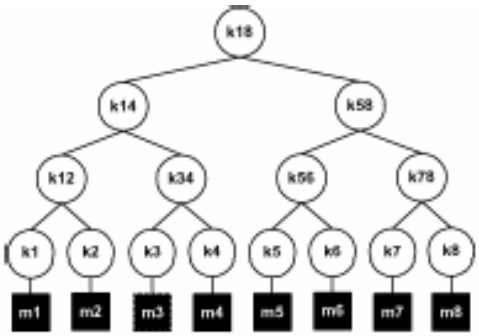
in care
h – este o functie one-way
n – este dimensiunea grupului
Ni – contributia membrului i.

Protocolul este urmatorul:
   N-1 membri difuzeaza in clar contributiile lor Ni.
   Liderul de grup (de ex U1) cripteaza N1 cu cheile publice ale celorlalti si difuzeaza mesajul.
   Fiecare membru (din cei n-1) decripteaza N1 si genereaza K.

Numarul de runde este 3.

D LKH – Distributed Logical Key Hierarchy

Propus de Rodeh.
Protocoluol nNu are un Group Controller care sa stie toate cheile. In schimb, cheile sunt stabilite de subarbori, in modul descris in continuare.
Fie subarborii L si R
Liderii m_l respectiv m_r
Cheile K_L respectiv K_R
Protocol:
m_l alege o noua cheie k_{LR} si o trimite lui m_r pe un canal securizat
m_l cripteaza k_{LR} cu k_L si difuzeaza rezultat tuturor membrilor subarbore L
m_r cripteaza k_{LR} cu k_R si difuzeaza rezultat tuturor membrilor subarbore R.

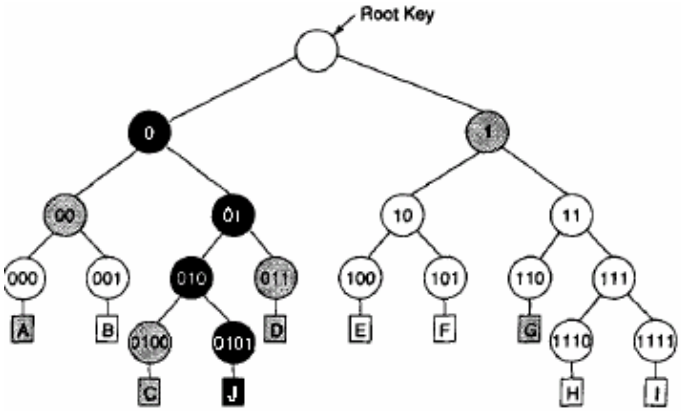


Exemplu: pentru arborele din figura:
membrii 1,2 stabilesc cheia **k12**
3,4 stabilesc cheia **k34**
1,2 si 3,4 stabilesc cheia **k14**
….
1,2,3,4 si 5,6,7,8 stabilesc cheia **k18**

D OFT – Distributed One-way Function Tree

Propus de Dondeti.
Protocolul foloseste OFT – One-way Function Tree, cu urmatoarele diferente:
Nu exista un Group Controller
Fiecare membru nou realizeaza urmatoarele operatii:
Controlul accesului
Generarea cheii proprii
Transmiterea cheii proprii "orbite" catre fratele sau.
Similar cu cazul OFT, fiecare membru cunoaste toate cheile din calea spre radacina si toate cheile "orbite" ale fratilor acestor noduri.
Numarul de runde este $\log_2 n$.

Prezentam in continuare un exemplu detaliat de generare a cheilor in care membrul J se alatura grupului, langa C.



C genereaza propria cheie 0101 si trimite cheia orbita 0101* lui C (prin conventie marcam cheia orbita cu *).
C schimba cheia 0100 si trimite 0100* lui J.
C calculeaza noile chei intermediare 010, 01, 0 si root folosind cheile orbite ale rudelor: 0101* (noua) si 011*,00*, 1* (care nu se schimba).
J primeste 0100* (de la C), calculeaza 010.
J trimite 010* lui D si primeste 011*; calculeaza apoi cheia 01.
J trimite 01* lui A si primeste 00*; calculeaza cheia 0.
J trimite 0* lui G; primeste 1*; calculeaza cheia din radacina.
A trimite lui B 01* criptata cu 00.
Similar, G trimite multicast 0* criptata cu cheia 1 catre E, F, H, I.

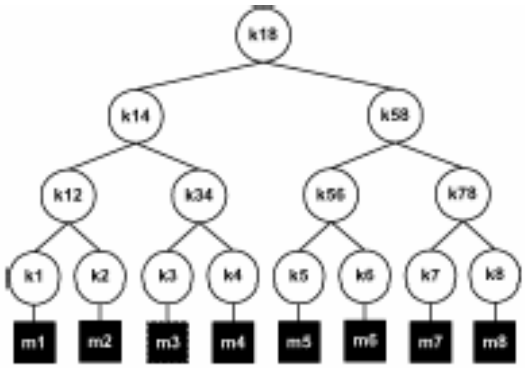
DH-LKH - Diffie-Hellman Logical Key Hierarchy

Propus de Perrig si Kim.
Protocolul foloseste LKH. Diferenta: membrii genereaza cheile in nivelul superior folosind algoritmul Diffie-Hellman (si nu one-way function):

$$k = a^{kl.kr} \bmod p$$

Exemplu (vezi figura urmatoare):
$k_{12} = a^{k1.k2} \bmod p$
$k_{34} = a^{k3.k4} \bmod p$
$k_{14} = a^{k12.k34} \bmod p$
…
$k_{58} = a^{k56.k78} \bmod p$
$k_{18} = a^{k14.k58} \bmod p$

Numarul de runde este $\log_2 n$.



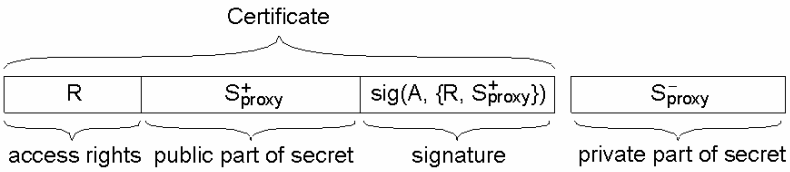
Delegarea drepturilor de acces

Sunt cunoscute doua abordari pentru delegarea drepturilor de la un utilizator sau proces la un alt proces care executa un serviciu in numele sau.

In cazul **certificatelor nominale**, Alice poate delega drepturile R lui Bob prin semnarea unui certificat de forma $[A, B, R]_A$. Daca Bob, la randul sau, trebuie sa delege o parte din drepturile R unui subserviciu, Charlie, Bob va trebui sa-i spuna lui Alice sa delege drepturi lui Charlie si sa-l puna pe Charlie sa o contacteze pe Alice pentru a capata drepturile.

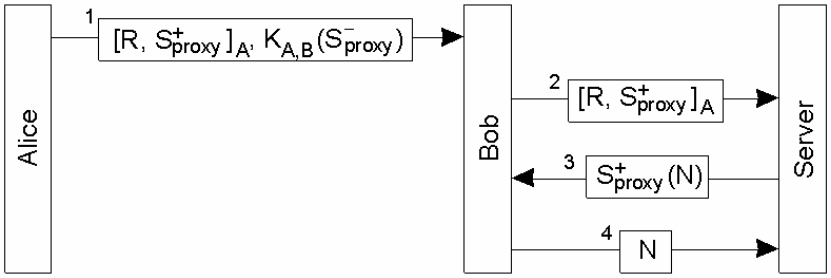
Certificatele **anonime** simplifica lucrurile, deoarece lasa pe Bob sa delege drepturile capatate de la Alice fara a o mai implica pe aceasta. Certificagtele anonime reclama insa protectia contra copierii ilegale. O solutie pentru aceasta protectie este data de certificatele proxy.

Proxy este un certificat care permite posesorului sa opereze cu aceleasi drepturi (sau drepturi mai restranse) ca garantul. Certificatul prin care Alice garanteaza cuiva drepturile R contine: drepturile R garantate, partea publica a unui secret S^+_{proxy} folosit pentru a autentifica detinatorul certificatului si semnatura lui Alice asupra certificatului care impiedica posesorul sa modifice drepturile acordate.



Pana aici, nimic despre identitatea procesului care va primi aceste drepturi. S^+_{proxy} – poate fi interpretat ca o “intrebare” publica al carui "raspuns" S^-_{proxy} il cunoaste doar Alice. Atunci cand garanteaza drepturile R lui Bob, Alice ii transmite proxy-ul si partea privata a secretului S^-_{proxy} in forma criptata cu o cheie partajata doar de Alice si Bob (mesajul 1 din figura urmatoare). Aceasta parte privata este folosita de Bob pentru a proba unui server ca este detinatorul legal al drepturilor de acces. Atunci cand acceseaza serverul, Bob transmite acestuia proxy-ul capatat de la Alice (mesajul 2). Serverul va folosi S^+_{proxy} pentru a cripta un numar N generat aleator si va transmite mesajul 3 lui Bob. Bob va

extrage numarul N folosind S^-_{proxy} si va trimite rezultatul catre serviciu (mesajul 4) dovedind ca este in posesia partii private a secretului (raspunsul).



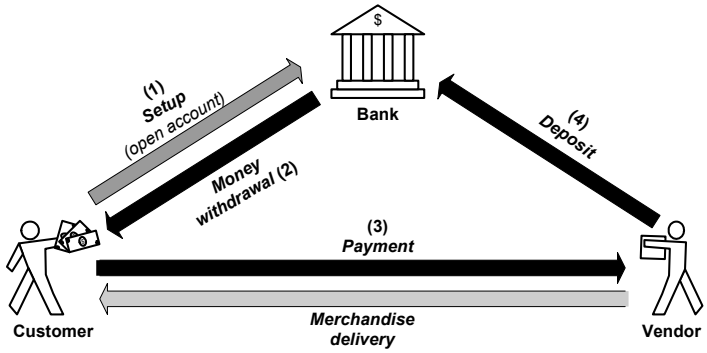
Pentru a pasa drepturile R unui alt proces, Bob trebuie sa trimita proxy-ul primit de la Alice si partea privata a secretului criptata cu o cheie partajata de Bob si procesul caruia i se transmit drepturile fara ca Alice sa mai participe la aceasta noua delegare de drepturi.

Ciclul de viata al cache-ului electonic

Una din problemele importante de securitate in multe aplicatii distribuite este **pastrarea intimitatii** (privacy) utilizatorilor. De exemplu, in aplicatiile de comert electronic, este important ca vanzatorul sau banca sa nu poata colecta si exploata informatii despre profilul cumparatorilor. Prezentam in continuare doua solutii: una pentru moneda electronica, alta pentru platile electronice cu carti de credit.

Ciclul de viata al monedei electronice cuprinde urmatoarele etape:

Clientul deschide un cont la banca.
Clientul extragere o suma de bani de la banca (cash electronic)
Clientul face o cumparatura si plateste cu cash electronic
Vanzatorul depunere banii la aceeasi banca.

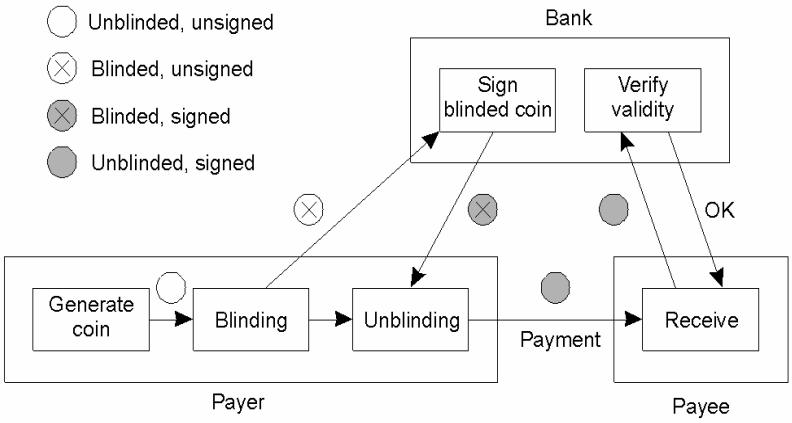


Semnaturi oarbe (Chaum 1982)

Problema este ca banca sa nu poata face legatura, prin monezile date clientului, intre acesta si cumparaturile facute. Solutia este cea a semnaturilor oarbe, propuse d Chaum. Consideram ca:

- cheile RSA ale bancii sunt - cea publica = (n,e) si cea privata = (n,d) iar
- *k* este un "secret" ales aleator de Alice

Protocolul este urmatorul: Alice genereaza o moneda de $10 notata m si cere bancii sa o semneze (fara ca banca sa o poata urmari ulterior).
Orbire - Alice genereaza k aleator, $0 \leq k \leq n-1$, GCD(n, k) =1 si cere bancii sa semneze o moneda electronica de $10 cu numarul $t = mk^e \bmod n$. Numarul mk^e este de fapt un fel de numar serial al monezii semnate.
Semnare - Banca semneaza t cu cheia privata d, obtinand t^d, scoate $10 din contul lui Alice si returneaza moneda semnata. De mentionat ca banca foloseste o semnatura specifica pentru monezile de $10.



"Dez"-orbire - Alice anuleaza orbirea prin calculul:

$$s = t^d / k \bmod n = (mk^e)^d / k \bmod n$$

$$s = m^d k^{ed-1} \bmod n = m^d \bmod n$$

Se obtine astfel o moneda m care este semnata de banca cu cheia sa privata si care nu include vreo "urma" care sa permita identificarea clientului caruia i-a dat moneda (banca a semnat o moneda cu "numar serial" t, in timp ce moneda pe care o foloseste acum Alice are "numarul serial" m).
Alice foloseste m^d ca moneda de $10 pentru plata lui Bob. Bob verifica impreuna cu banca validitatea monedei si accepta plata.
Problema: nu releva identitatea lui Alice in cazul in care ea incearca sa foloseasca de mai multe ori moneda, deoarece banca nu poate face legatura intre moneda m si platitor.

Semnaturi duale

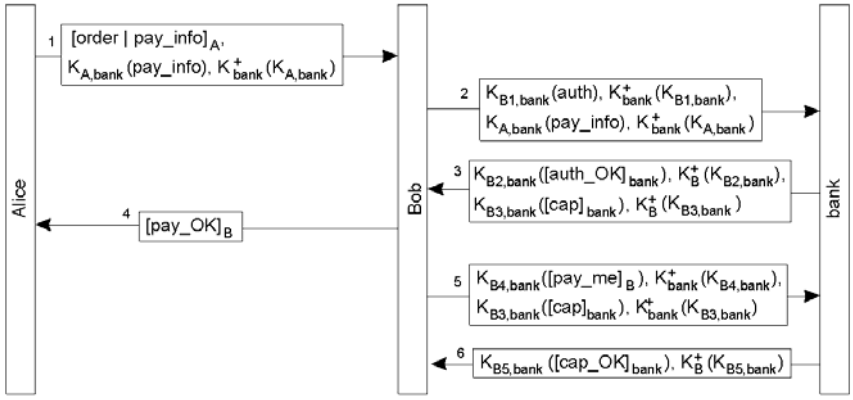
Al doilea mecanism este cel al semnaturilor duale, **dual signature** (aplicate de Alice si transmise lui Bob). Sa consideram ca Alice face o cumparatura de la Bob folosind cardul de credit. Ea trebuie sa trimita lui Bob ordinul impreuna cu informatia de card, semnate impreuna. Bob ar da bancii informatia de credit card si ar retine ordinul de plata pe care trebuie sa-l satisfaca. Pentru ca Bob si banca sa poata verifica semnatura, trebuie ca ambii sa primeasca in clar atat ordinul cat si informatia de card. Banca ar afla astfel ce a cumparat Alice. Semnatura duala permite ocolirea acestui neajuns.
Daca m1 denota ordinul si m2 este informatia de plata, Alice poate trimite lui Bob un mesaj de forma

$$[m1 \mid m2]_A = [m1, H(m2), H(concat(m1, m2)), K^{-A}(H(concat(m1, m2)))]$$

care reprezinta semnatura duala a mesajelor m1 si m2. Mesajul contine ordinul m1 in clar (Bob il poate intelege), doua rezumate, unul pentru ordinul m1 si al doilea pentru m1 si m2 concatenate, precum si semnatura lui Alice pe rezumatul concatenarii. Bob nu poate citi m2 dar este sigur ca Alice a pus semnatura pe intregul mesaj.

Criptografia in SET.

Semnatura duala este folosita in protocolul SET (Secure Electronic Transactions).



Sunt folosite urmatoarele notatii:

$K_{A,bank}$, $K_{Bi,bank}$ – chei de sesiune
K^-_{bank} / K^+_{bank} – chei privata / publica
auth – cerere de autorizare
pay_me – cerere de plata
auth_OK – mesaj autorizare
cap – mesaj captura
pay_OK – confirmarea platii

Protocolul este urmatorul:

1 – Alice transmite ordinul si informatia de plata, semnate dual; in plus transmite informatia de plata pentru banca, criptata cu o cheie de sesiune intre Alice si banca, si cheia de sesiune criptata cu cheia publica a bancii.
2 – Bob retine ordinul si paseaza bancii o cerere de autorizare cu informatia de plata; se foloseste acelasi mecanism de criptare a informatiei cu o cheie de sesiune care este transmisa criptat cu cheia publica a receptorului.
3 – Dupa verificarea informatiei de plata, banca trimite mesajul de autorizare 3, impreuna cu un mesaj captura ce va fi folosit de Bob pentru a cere ulterior plata de la banca.
4 – Bob confirma lui Alice validitatea platii.
5 – Bob trimite bancii mesajul 5 de cerere de plata insotit de captura.
6 – Banca efectueaza plata si transmite mesajul 6 de confirmare a platii.

Bibliografie

SANDRO RAFAELI AND DAVID HUTCHISON. A Survey of Key Management for Secure Group Communication, ACM Computing Surveys, Vol. 35, No. 3, September 2003, pp. 309–329.